

Informe Tecnico

Servidor MCP para Gmail

Practica Evaluable - Unidad 6: Model Context Protocol

Alumno:	Guillermo Casaus Lopez
Curso:	INSO 4B
Asignatura:	Aprendizaje Automatico II
Tecnologias:	FastMCP, Gmail API, OAuth 2.0, Claude Desktop
Repositorio:	GitHub - aprendizaje-automatico-II / practicas / unidad6_practica

Resumen

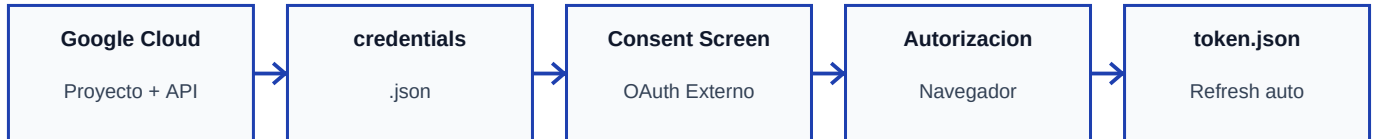
El presente informe documenta el desarrollo e implementacion de un servidor MCP (Model Context Protocol) que integra la API de Gmail con Claude Desktop. El sistema permite al modelo de lenguaje listar, enviar y consultar correos electronicos a traves de herramientas MCP, un recurso de perfil y una plantilla de redaccion. Se documenta el flujo de autentificacion OAuth 2.0, los componentes implementados, las dificultades encontradas durante el desarrollo y las mejoras propuestas para versiones futuras.

Contenido del informe

1. Flujo OAuth 2.0 implementado
2. Componentes del servidor MCP
3. Dificultades encontradas durante el desarrollo
4. Posibles mejoras y extensiones

1. Flujo OAuth 2.0 implementado

La autenticacion con la API de Gmail se basa en el protocolo OAuth 2.0, estandar de delegacion de acceso que permite a la aplicacion operar sobre la cuenta del usuario sin manejar directamente su contrasena. El flujo implementado sigue cinco fases:



1.1 Configuracion en Google Cloud Console

Se crea un proyecto dedicado en Google Cloud Console y se habilita la Gmail API. A continuacion se configura la pantalla de consentimiento OAuth con tipo 'Externo', se anade la cuenta personal como usuario de prueba, y se generan credenciales de tipo 'Aplicacion de escritorio' que producen el fichero credentials.json. Los scopes requeridos son gmail.readonly, gmail.send y gmail.modify.

1.2 Autorizacion inicial (primera ejecucion)

La primera vez que se ejecuta el servidor, la funcion `get_gmail_service()` detecta la ausencia de token.json y lanza `InstalledAppFlow.run_local_server(port=0)`. Esto abre el navegador del sistema mostrando la pantalla de consentimiento de Google. El usuario acepta los permisos y Google devuelve un codigo de autorizacion a un servidor local temporal que la libreria `google-auth-oauthlib` levanta automaticamente.

1.3 Intercambio de codigo por tokens

La libreria intercambia el codigo de autorizacion por un access token (valido ~1 hora) y un refresh token (larga duracion). Ambos se persisten en token.json para que las ejecuciones siguientes no requieran intervencion del usuario.

1.4 Renovacion automatica del token

En cada llamada a `get_gmail_service()`, el codigo verifica si las credenciales han expirado. Si `creds.expired` y `creds.refresh_token` estan disponibles, ejecuta `creds.refresh(Request())` para obtener un nuevo access token sin intervencion humana. Esto garantiza que el servidor MCP funciona de forma desatendida una vez realizada la primera autenticacion.

1.5 Seguridad

Los ficheros credentials.json y token.json contienen informacion sensible y estan excluidos del repositorio mediante .gitignore. El evaluador debe configurar sus propias credenciales para reproducir el entorno. Los scopes solicitados siguen el principio de minimo privilegio: unicamente los permisos estrictamente necesarios para las operaciones implementadas.

2. Componentes del servidor MCP

El servidor se implementa con FastMCP, framework que simplifica la exposicion de herramientas, recursos y prompts conforme al protocolo Model Context Protocol (MCP). MCP es un estandar abierto de Anthropic que define como los modelos de lenguaje pueden interactuar con sistemas externos de forma estructurada y segura.

2.1 Tools (Herramientas)

Las tools son funciones que Claude puede invocar activamente en respuesta al lenguaje natural del usuario. Claude Desktop las descubre automaticamente al iniciar el servidor.

list_emails	Lista los N emails mas recientes de la bandeja de entrada (INBOX). Consulta los metadatos de cada mensaje (From, Subject, Date) usando el endpoint messages.list + messages.get con format='metadata'. Parametro: max_results (por defecto 10).
send_email	Compone un mensaje MIME en texto plano, lo codifica en Base64 URL-safe y lo envia a traves del endpoint messages.send de la Gmail API. Parametros: to (destinatario), subject (asunto), body (cuerpo). Devuelve confirmacion con el ID del mensaje generado por Gmail.
get_gmail_profile	Obtiene el perfil del usuario de Gmail: direccion de email, total de mensajes e hilos. Implementada para que Claude la utilice de forma automatica cuando se pregunta por el perfil en lenguaje natural. Complementa al resource gmail://profile (ver seccion 2.2).

2.2 Resource (Recurso)

Los resources MCP exponen datos como URIs que los clientes pueden consultar. A diferencia de las tools, no son invocados automaticamente por el modelo; requieren que el cliente los solicite explicitamente por URI.

gmail://profile	Expone el perfil del usuario (email, messagesTotal, threadsTotal) bajo el URI gmail://profile. Decorado con @mcp.resource('gmail://profile'). Accesible desde clientes MCP que soporten el mecanismo de resources.
-----------------	--

2.3 Prompt (Plantilla)

Los prompts MCP son plantillas de instrucciones parametrizadas que el cliente puede inyectar en la conversacion para guiar el comportamiento del modelo.

redactar_email	Genera un prompt estructurado para redactar un email profesional. Parametros: destinatario, tema y tono (por defecto 'profesional'). El prompt resultante indica al modelo incluir saludo, cuerpo y despedida adecuados al tono elegido.
----------------	--

2.4 Estructura del proyecto

Aprendizaje Automatico II - Unidad 6	Pagina 2
--------------------------------------	----------

unidad6_practica/

gmail_mcp_server.py # Servidor MCP: tools, resource y prompt

requirements.txt # Dependencias del proyecto

claude_desktop_config.json # Configuracion de Claude Desktop (excluir de git)

credentials.json # Credenciales OAuth de Google Cloud (excluir de git)

token.json # Token OAuth generado tras autenticacion (excluir de git)

.gitignore # Excluye credentials.json y token.json

capturas/ # Evidencia de las pruebas realizadas

2.5 Integracion con Claude Desktop

Claude Desktop carga el servidor MCP a traves del fichero `claude_desktop_config.json`, que especifica la ruta absoluta al interprete Python del entorno virtual y al script `gmail_mcp_server.py`. Al arrancar Claude Desktop, lanza el proceso del servidor y expone sus tools al modelo. El servidor ejecuta `get_gmail_service()` en el bloque `__main__` para forzar la autenticacion OAuth antes de arrancar, de modo que el flujo de consentimiento ocurre en una ejecucion manual previa.

3. Dificultades encontradas

Dificultad 1 Resources MCP no visibles automaticamente en Claude Desktop

La principal dificultad fue comprobar que Claude Desktop expone las tools al modelo de forma nativa, pero los resources MCP siguen un mecanismo diferente: requieren que el cliente los liste y los inyecte explicitamente en el contexto. Al preguntar en lenguaje natural '¿Cual es mi perfil de Gmail?', Claude respondia que solo disponia de dos funciones (list_emails y send_email), ignorando por completo el resource gmail://profile.

Solucion adoptada: se anadio una tool adicional get_gmail_profile() con la misma logica que el resource. Claude la detecta y usa automaticamente para responder preguntas sobre el perfil, mientras el resource permanece en el codigo para cumplir con los requisitos del enunciado.

Impacto: Alto - impidio completar la Prueba 3 en el primer intento **Solucion:** Anadir tool equivalente junto al resource existente

Dificultad 2 Rutas absolutas en la configuracion de Claude Desktop

La configuracion del servidor MCP en claude_desktop_config.json requiere rutas absolutas al interprete Python y al script. En entornos con entorno virtual (.venv), fue necesario apuntar exactamente al ejecutable dentro del venv para que las dependencias (fastmcp, google-auth-oauthlib, google-api-python-client) estuviesen disponibles. Usar el Python del sistema provocaba errores de importacion al arrancar el servidor.

Impacto: Medio - el servidor no arrancaba hasta encontrar la ruta correcta **Solucion:** Usar la ruta completa al python.exe del .venv en la configuracion

Dificultad 3 Autenticacion OAuth en contexto de servidor MCP

El servidor MCP es lanzado por Claude Desktop como proceso hijo sin consola interactiva. El flujo OAuth requiere abrir el navegador y esperar la respuesta del usuario, lo que no es posible durante el arranque automatico del servidor. Si token.json no existia, el servidor quedaba colgado esperando interaccion.

Solucion: se anadio una llamada a get_gmail_service() en el bloque __main__ del script. Esto obliga al usuario a ejecutar el script manualmente una primera vez (python gmail_mcp_server.py) para completar el flujo OAuth y generar token.json. Las ejecuciones posteriores desde Claude Desktop usan el token cacheado.

Impacto: Medio - requirio un paso de configuracion inicial adicional **Solucion:** Ejecucion manual previa del script para generar token

4. Posibles mejoras y extensiones

A continuacion se describen las mejoras mas relevantes que se implementarían en una version futura del servidor para aumentar su cobertura funcional y robustez en un entorno de uso real.

Mejora 1

Lectura del cuerpo completo de un email

La tool `list_emails` actual solo recupera metadatos (remitente, asunto, fecha). Una mejora directa seria anadir una tool `read_email(id)` que descargue el mensaje completo con `format='full'`, decodifique el payload en Base64 y devuelva el cuerpo en texto plano o HTML limpio. Esto permitiría a Claude resumir correos, responder a hilos de conversacion o extraer informacion de mensajes concretos.

Impacto:

Alto - amplia significativamente la utilidad del asistente

Esfuerzo:

Bajo - ~20 lineas adicionales reutilizando `get_gmail_ser`

Mejora 2

Busqueda de emails por criterio (`search_emails`)

La Gmail API soporta la misma sintaxis de busqueda avanzada que el cliente web: `from:`, `to:`, `subject:`, `after:`, `before:`, `has:attachment`, etc. Implementar una tool `search_emails(query)` que acepte una cadena de busqueda libre y devuelva los resultados filtrados multiplicaria la utilidad del servidor. Claude podria resolver consultas como 'Busca los emails de facturas del mes pasado' de forma natural.

Impacto:

Alto - caso de uso muy frecuente en asistentes de correo

Esfuerzo:

Bajo - el endpoint `messages.list` acepta el parametro `q`

Mejora 3

Gestion de etiquetas y estado de mensajes

Implementar tools para marcar mensajes como leidos/no leidos (`messages.modify`), mover mensajes entre carpetas, archivarlos o eliminarlos anadaria capacidades de gestion completa del correo. Combinado con `list_emails` y `search_emails`, permitiría a Claude actuar como asistente proactivo de organizacion: por ejemplo, 'Archiva todos los emails de newsletters de la ultima semana'.

Impacto:

Alto - convierte el servidor en un gestor de correo completo

Esfuerzo:

Medio - requiere scope `gmail.modify` (ya configurado) y

Mejora 4

Soporte para emails HTML y adjuntos en `send_email`

El envio actual se limita a texto plano (`MIMEText`). Extender `send_email` con partes MIME adicionales permitiría enviar emails con formato HTML (`MIMEMultipart` con parte `text/html`) y adjuntar ficheros codificados en Base64. Esto cubriría casos de uso profesionales como el envio de informes, facturas o presentaciones directamente desde Claude Desktop.

Impacto:

Medio - mejora la calidad de los emails enviados

Esfuerzo:

Medio - requiere refactorizar la construccion del mensaj

Valoracion global

El servidor MCP desarrollado cumple con todos los requisitos funcionales de la practica: autenticacion OAuth 2.0 correctamente implementada, dos tools funcionales (`list_emails`, `send_email`), un resource MCP (`gmail://profile`), una tool auxiliar (`get_gmail_profile`) para compatibilidad con Claude Desktop, y una plantilla de redaccion (`redactar_email`). Las cuatro pruebas obligatorias han sido ejecutadas satisfactoriamente. Las mejoras propuestas permitirían evolucionar el servidor hacia un asistente de correo completo.